

ErpOne — Developer Guide

Adding a Menu Entry & Building a Form (end-to-end vertical slice)

Canonical example: `Brand` master. Applies to any CRUD feature. Generated 2026-07-14

0. The vertical slice

A new feature touches every layer. Build in this order:

1. **Domain** — entity + EF config + migration.
2. **Application** — DTOs, service interface, FluentValidation validators.
3. **Infrastructure** — service implementation + DI registration.
4. **Web** — Index (list), Form (create/edit), optional Detail page.
5. **Menu & permissions** — register in `AppMenus.cs`.
6. **Tests** — integration test for the service.

Place files in the subfolder matching the menu header (Master / Transactions / Finance / ...) in each project. Keep the existing namespace (folders are organizational only).

Part A — Register the menu & permissions

A1 Add the resource in `AppMenus.cs`

Add an `AppResource` to the correct `ResourceGroup`. The key drives route, permission, and icon by convention.

```
// Web/Authorization/AppMenus.cs → inside the "Master" group
new("master.brands", "Brand", "bi-bookmark-star-fill", CRUD),
// key          label      bootstrap icon      actions preset
```

Action presets already defined: `CRUD` = index/create/edit/delete; `ViewOnly`, `ViewCreate`, `ReportActions` (index+export), and document presets (with approve/post/void). This auto-creates permissions `master.brands.index|create|edit|delete`.

A2 Routing convention (and overrides)

`NavMenuBuilder` derives the link automatically:

- Href = `key.Replace('.', '/')` → `master.brands` ⇒ `/master/brands`.
- Visibility gate = `{key}.index`.
- Icon = declared icon minus `-fill`.

Only if the route/action is irregular, add an entry to the `Overrides` dictionary:

```
// e.g. category route differs, POS points to the create action + extra link
["master.categories"] = new(Href: "master/product-categories"),
["cashier.pos"]       = new(Href: "cashier/pos", Action: "create", Extra: [ ... ]),
```

A3 Grant the permission

`BootstrapSeeder` grants *all* `AppMenus.AllPermissions` to the admin role on startup (idempotent). No code needed.

Manual step after deploy: restart the app and **sign out / sign in** so the admin's permission claims refresh. Configure non-admin roles under *Settings* → *Role*.

Part B — Domain layer

B1 Entity

```
// Domain/Entities/Master/Brand.cs - namespace ErpOne.Domain.Entities
public class Brand : AuditableEntity // gives CreatedAt/CreatedBy/...
{
    public int Id { get; private set; }
    public string Code { get; private set; } = default!;
    public string Name { get; private set; } = default!;
    public string? Description { get; private set; }

    private Brand() { } // EF
    public Brand(string code, string name, string? description)
    {
        if (string.IsNullOrEmpty(code)) throw new ArgumentException("Code required.",
nameof(code));
        Code = code.Trim(); Name = name.Trim(); Description = description;
    }
    public void Update(string code, string name, string? description) { /* mutate + validate */ }
}
```

Rich domain model: private setters, behavior methods, guard clauses. Enums (statuses) go in their own file in the same folder.

B2 EF configuration + migration

```
// Infrastructure/Persistence/AppDbContext.cs
public DbSet<Brand> Brands => Set<Brand>();
// in OnModelCreating:
modelBuilder.Entity<Brand>(e => {
    e.ToTable("M_Brands"); // M_ master / T_ transaction prefix
    e.HasKey(x => x.Id);
    e.Property(x => x.Code).HasMaxLength(20).IsRequired();
    e.HasIndex(x => x.Code).IsUnique();
    e.Property(x => x.Name).HasMaxLength(100).IsRequired();
});
```

```
dotnet ef migrations add AddBrand --project src/ErpOne.Infrastructure --startup-project
src/ErpOne.Web
dotnet ef database update --project src/ErpOne.Infrastructure --startup-project
src/ErpOne.Web
```

Part C — Application layer

C1 DTOs

```
// Application/Master/Brands/BrandDtos.cs - namespace ErpOne.Application.Brands
public record BrandDto(int Id, string Code, string Name, string? Description, DateTime CreatedAt,
string? CreatedBy);
public record CreateBrandRequest(string Code, string Name, string? Description);
public record UpdateBrandRequest(string Code, string Name, string? Description);
```

C2 Service interface

```
public interface IBrandService
{
    Task<IReadOnlyList<BrandDto>> GetAllAsync(CancellationToken ct = default);
    Task<PagedResult<BrandDto>> GetPagedAsync(int page, int pageSize, string? search = null,
CancellationTokens ct = default);
    Task<BrandDto?> GetByIdAsync(int id, CancellationToken ct = default);
    Task<BrandDto> CreateAsync(CreateBrandRequest request, CancellationToken ct = default);
    Task<bool> UpdateAsync(int id, UpdateBrandRequest request, CancellationToken ct = default);
    Task<bool> DeleteAsync(int id, CancellationToken ct = default);
}
```

C3 Validators (FluentValidation)

```
public class CreateBrandValidator : AbstractValidator<CreateBrandRequest>
{
    public CreateBrandValidator()
    {
        RuleFor(x => x.Code).NotEmpty().MaximumLength(20)
            .Matches("^[A-Za-z0-9-]+$").WithMessage("Code may only contain letters, numbers, and dashes.");
        RuleFor(x => x.Name).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Description).MaximumLength(300);
    }
}
// + UpdateBrandValidator. Auto-registered via
AddValidatorsFromAssemblyContaining<CreateProductValidator>().
```

Part D — Infrastructure layer

D1 Service implementation

```
// Infrastructure/Services/Master/BrandService.cs - namespace ErpOne.Infrastructure.Services
public class BrandService(
    AppDbContext db,
    IValidator<CreateBrandRequest> createValidator,
    IValidator<UpdateBrandRequest> updateValidator) : IBrandService // primary-ctor DI
{
    public async Task<PagedResult<BrandDto>> GetPagedAsync(int page, int pageSize, string? search = null, CancellationToken ct = default)
    {
        page = Math.Max(1, page); pageSize = Math.Clamp(pageSize, 1, 200);
        var q = db.Brands.AsNoTracking();
        if (!string.IsNullOrEmpty(search))
            q = q.Where(x => x.Name.Contains(search) || x.Code.Contains(search));
        var total = await q.CountAsync(ct);
        var items = await q.OrderBy(x => x.Name).Skip((page-1)*pageSize).Take(pageSize)
            .Select(x => ToDto(x)).ToListAsync(ct);
        return new PagedResult<BrandDto>(items, total, page, pageSize);
    }

    public async Task<BrandDto> CreateAsync(CreateBrandRequest req, CancellationToken ct = default)
    {
        await createValidator.ValidateAndThrowAsync(req, ct); // throws ValidationException
        var e = new Brand(req.Code, req.Name, req.Description);
        db.Brands.Add(e); await db.SaveChangesAsync(ct);
        return ToDto(e);
    }

    // GetAll/GetById/Update/Delete similar; always AsNoTracking for reads.
    private static BrandDto ToDto(Brand x) => new(x.Id, x.Code, x.Name, x.Description, x.CreatedAt, x.CreatedBy);
}
```

D2 Register in DI

```
// Infrastructure/DependencyInjection.cs
services.AddScoped<IBrandService, BrandService>();
```

Part E — Web: Index (list) page

Use the `.pi` master-index pattern: crumbs + KPIs (optional) + toolbar (search / filters / chips) + card table + `<Pager>`. Gate route with the `.index` policy.

```
// Web/Components/Pages/Master/Brands/BrandIndex.razor
@page "/master/brands"
@attribute [Authorize(Policy = "master.brands.index")]
@rendermode InteractiveServer
@Inject IBrandService BrandService

<div class="pi">
  <div class="pi-head"> /* crumbs + title + New button (AuthorizeView create) */ </div>
  <div class="toolbar">
    <div class="search"><i class="bi bi-search"></i>
      <input placeholder="Search..." @bind="_search" @bind:event="oninput" @onkeyup="ReloadAsync" />
    </div>
  </div>
  @* card + <table> over _page.Items, then <Pager Page= TotalPages= OnPageChanged= /> *@
</div>

@code {
  private const int PageSize = 15;
  private PagedResult<BrandDto>? _page; private string? _search; private int _currentPage = 1;
  protected override Task OnInitializedAsync() => LoadAsync();
  private async Task LoadAsync() => _page = await BrandService.GetPagedAsync(_currentPage, PageSize, _search);
  private async Task ReloadAsync() { _currentPage = 1; await LoadAsync(); }
  private async Task GoToPageAsync(int p) { _currentPage = p; await LoadAsync(); }
}
```

For status filters use chips: `<div class="chips"><button class="chip @(on?"on":"","")" @onclick="...">...</button></div>` (see Stock Levels for a working example).

Part F — Web: Form (create/edit) page

Master forms use the `.cf` container (card + responsive grid). One route handles both create and edit. Re-check permission in `OnInitializedAsync`. Map `ValidationException` errors back to per-field messages.

```
// Web/Components/Pages/Master/Brands/BrandForm.razor
@page "/master/brands/new"
@page "/master/brands/{Id:int}/edit"
@attribute [Authorize]
@rendermode InteractiveServer
@using FluentValidation
@inject IBrandService BrandService
@inject IAuthorizationService Auth
@inject NavigationManager Nav

<div class="cf">
    <div class="cf-top"><div class="crumbs"> ... </div></div>
    <div class="cf-wrap">
        <section class="card">
            <div class="card-h"><span class="hd-ic"><i class="bi bi-award"></i></span>
                <div class="hd-tx"><h2>Brand information</h2><p>...</p></div></div>
            <div class="card-b">
                <div class="grid">
                    <div class="f c4"><label class="fl">Code <span class="req">*</span></label>
                        <input class="ctl mono up @(_codeError!=null?"bad":"" )" @bind="_code"
@bind:event="oninput" />
                        <div class="err-txt">@_codeError</div></div>
                    <div class="f c8"> ... Name ... </div>
                    <div class="f c12"> ... Description (textarea) ... </div>
                </div>
            </div>
        </section>
        <div class="pf-footer"><div class="in">
            <a class="btn btn-ghost" href="/master/brands">Cancel</a>
            <button class="btn btn-primary" @onclick="SaveAsync" disabled="@_saving">Save brand</button>
        </div></div>
    </div>
</div>

@code {
    [Parameter] public int? Id { get; set; }
    [CascadingParameter] private Task<AuthenticationState> AuthStateTask { get; set; } = default!;
    private string _code="", _name=""; private string? _description, _error, _codeError, _nameError;
    private bool _loading=true, _saving, _notFound;

    protected override async Task OnInitializedAsync() {
        var user = (await AuthStateTask).User;
        var perm = Id is null ? AppMenus.Perm("master.brands","create") :
AppMenus.Perm("master.brands","edit");
        if (!(await Auth.AuthorizeAsync(user, perm)).Succeeded) { Nav.NavigateTo("/master/brands");
return; }
        if (Id is int id) { var b = await BrandService.GetByIdAsync(id);
            if (b is null) _notFound = true; else { _code=b.Code; _name=b.Name; _description=b.Description;
} }
        _loading = false;
    }

    private async Task SaveAsync() {
        _error=_codeError=_nameError=null; _saving=true;
        try {
```

```

        if (Id is int id) await BrandService.UpdateAsync(id, new
UpdateBrandRequest(_code, _name, _description));
        else
            await BrandService.CreateAsync(new
CreateBrandRequest(_code, _name, _description));
        Nav.NavigateTo("/master/brands");
    }
    catch (ValidationException ex) {
        // map field errors
        foreach (var e in ex.Errors) {
            if (e.PropertyName == nameof(CreateBrandRequest.Code)) _codeError = e.ErrorMessage;
            else if (e.PropertyName == nameof(CreateBrandRequest.Name)) _nameError = e.ErrorMessage;
            else _error = e.ErrorMessage;
        }
    }
    finally { _saving = false; }
}
}

```

Form grid & control classes (.cf)

Class	Purpose
<code>.grid</code> + <code>.f</code> <code>.cN</code>	12-column responsive grid; a field spans <code>c4</code> / <code>c8</code> / <code>c12</code> etc.
<code>.fl</code> / <code>.req</code>	Field label / required asterisk.
<code>.ctl</code> (+ <code>.mono</code> <code>.up</code> <code>.bad</code>)	Input/select/textarea; <code>.up</code> uppercases, <code>.bad</code> = error border.
<code>.err-txt</code> / <code>.cnt</code>	Inline field error / character counter.
<code>.pf-footer</code> <code>.in</code>	Sticky action bar (Cancel / Save).

Transaction forms (with line-item tables) use `<div class="pf pf--form">` and a `.items` table instead of `.cf`. Detail pages use `<div class="pf pf--detail">` with an info-grid + `.items` + status `.badge`. See `PoForm` / `PoDetail`.

Part G — Tests

Integration test the service against SQLite. Fixture builds the schema; seed via services or the `DbContext`.

```
public class BrandServiceTests : IClassFixture<CustomWebApplicationFactory>
{
    private readonly CustomWebApplicationFactory _f;
    public BrandServiceTests(CustomWebApplicationFactory f) { _f = f; _f.InitializeDatabase(); }

    [Fact] public async Task Create_then_get_roundtrips() {
        using var scope = _f.Services.CreateScope();
        var svc = scope.ServiceProvider.GetRequiredService<IBrandService>();
        var dto = await svc.CreateAsync(new CreateBrandRequest("NKE", "Nike", null));
        Assert.Equal("Nike", (await svc.GetByIdAsync(dto.Id))!.Name);
    }
}
```

SQLite is one in-memory connection shared across a test class (isolated per class). Avoid SQL-Server-only EF translations. Stop the running app before `dotnet test` (tests reference `ErpOne.Web`).

Part H — Definition of done (checklist)

- ☐ Entity + EF config + migration applied.
- ☐ DTOs, service interface, validators, service impl, DI registration.
- ☐ `AppMenus.cs` resource added (+ `Overrides` if route/action irregular).
- ☐ Index page (`.pi`), Form page (`.cf` / `.pf-form`), Detail page (`.pf-detail`) if needed; all with `[Authorize(Policy)]` + `AuthorizeView` on actions.
- ☐ Integration test green; `dotnet build ErpOne.slnx` = 0 errors.
- ☐ Restart + sign out/in so admin gets the new permission.
- ☐ New file placed in the correct menu-header subfolder (namespace unchanged).